HPC Performance Analysis: Cache Saturation and Thread Scalability in NPB-CG

Sergi J.
https://github.com/imserez

***Abstract*—This report analyzes the performance of the Conjugate Gradient (CG) benchmark from the NAS suite on an HPC node equipped with an Intel Xeon E5603 processor. Using dynamic binary instrumentation (Intel PIN), the results demonstrate that the main bottleneck is caused by a high cache miss rate, driven by the low spatial locality of sparse matrix operations.**

***Index Terms*—HPC, OpenMP, Cache Thrashing, Intel PIN, Sparse Matrix.**

I. Benchmark Overview (Section 1)

The selected benchmark is NPB3.2 CG. It computes the conjugate gradient to approximate the smallest eigenvalue of a sparse, symmetric, and positive definite matrix. This type of computation is common in applications involving unstructured meshes, such as computational fluid dynamics simulations.

The benchmark is characterized by irregular memory access: the processor must access data that is not contiguous in memory. It consists of an iterative loop where the most expensive operation is the multiplication of a sparse matrix by a vector, followed by vector update operations. The memory access pattern is unpredictable, resulting in a low cache hit rate and high memory latency.

A. Parallelization and Parameters

Parallelization is implemented using OpenMP. Each thread is responsible for computing a subset of matrix rows. The problem size can be configured (Class S, W, A, B), which determines the matrix dimensions and number of elements. The number of threads can also be varied.

CG provides loop unrolling options (factors of 2 and 8). With unrolling factor 8, instead of performing 8 individual iterations, a single iteration performs 8 computations, reducing branch overhead. However, this may lead to increased instruction footprint that does not fit well in the instruction cache. The benchmark parameters are summarized in Table I.

Table I
Params per class of the CG Benchmark

Paràmetre	Class S	Class W	Class A	Class B
No. rows	1400	7000	14000	75000
No. nonzero	7	8	11	13
No. iterations	15	15	15	75
No. eig. shift	10	12	20	60
Random pairs	2^{24}	2^{25}	2^{28}	2^{30}

II. Performance Measurement (Section 2)

A. Instrumentation and Experimental Setup

Performance measurements were obtained using Intel PIN, which dynamically injects instrumentation code into the program to collect execution metrics.

B. Use of inscount_tls

The `inscount_tls` tool (Thread Local Storage) was used instead of `inscount`, which relies on a global counter. In a multithreaded context, this could lead to race conditions, as it is not thread-safe. With `_tls`, each thread maintains its own independent instruction counter.

C. Processor Architecture Analysis

The Intel Xeon E5603 is based on the Westmere microarchitecture. It consists of 4 physical cores without Hyper-Threading, meaning a maximum of 4 concurrent threads. Its cache hierarchy includes a private 32KB L1 (data/instruction), a private 256KB L2, and a shared 4MB L3 cache.

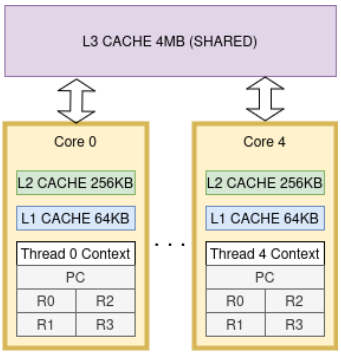

Figure 1. Overview of the 4-core westmere architecture

D. Results Analysis

Figure 2 presents execution time and throughput (MOP/s) as the number of threads varies across benchmark classes. Performance degradation when moving from 4 to 8 threads partially illustrates Amdahl's Law, as not all code is parallelizable. However, the dominant factor limiting scalability is memory bandwidth saturation due to cache misses.

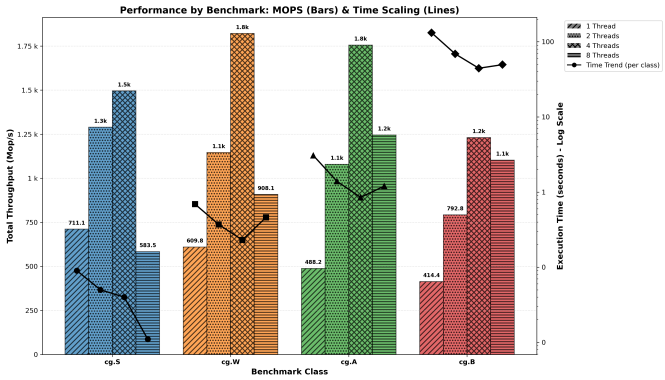

Figure 2. MOP/s and execution time

E. Impact of Multithreading

Multithreading introduces additional complexity in cache behavior, increasing miss rates and coherence overhead. Since the Xeon E5603 does not support Hyper-Threading, only one thread per physical core is available. When increasing from 4 to 8 threads, the operating system performs context switching, introducing overhead due to state saving and restoration. This performance degradation is visible in Figure 2. Load balancing appears adequate, as shown in Figure 3, indicating that imbalance is not the primary issue.

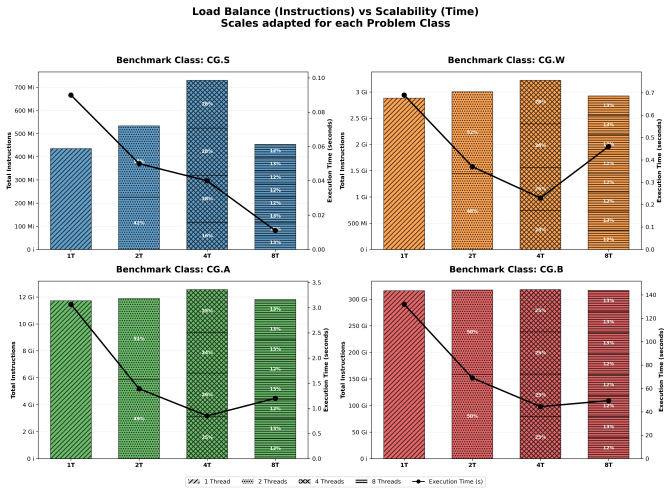

Figure 3. Total instructions / instructions per thread and time of execution

F. Performance and Instruction Count

As the number of threads increases, the total number of executed instructions also increases, despite running the same workload. This is due to multithreading overhead. The observed reduction in total instructions at 8 threads can also result from threads being stalled or blocked, rather than actively executing, in addition to potential operating system context management.

G. Loop Unrolling

Loop unrolling reduces branch overhead but does not significantly improve performance in this benchmark, as the primary bottleneck lies in memory access.

H. Cache Behavior

When data is not found in cache, the processor must fetch it from a lower level in the memory hierarchy. Cache lines (typically 64 bytes) are loaded to exploit spatial locality. However, sparse matrices do not benefit from this mechanism. The processor cannot effectively predict future accesses, leading to poor cache utilization across all levels (L1, L2, L3).

With multiple threads issuing irregular memory requests, memory bandwidth becomes saturated, forcing frequent accesses to main memory. This results in the well-known Memory Wall bottleneck.

As shown in Table II, this leads to high cache miss rates.

Table II
L1 Miss Rate (%) per Size and Threads

Class	1 Th.	2 Th.	4 Th.	8 Th.
S	13.92%	12.74%	16.08%	16.73%
W	26.53%	26.08%	30.36%	30.05%
A	28.63%	28.50%	32.57%	32.64%

I. AMAT Analysis

Miss rate alone can be misleading without considering its latency cost. A more accurate metric is the Average Memory Access Time (AMAT):

$$\text{AMAT} = \text{Hit Time} + (\text{Miss Rate} \times \text{Miss Penalty}) \quad (1)$$

In this architecture, L1 hit time is approximately 4 cycles, while the miss penalty (main memory access) can exceed 200 cycles.

The computed AMAT confirms that the processor spends most of its time stalled, waiting for memory accesses.

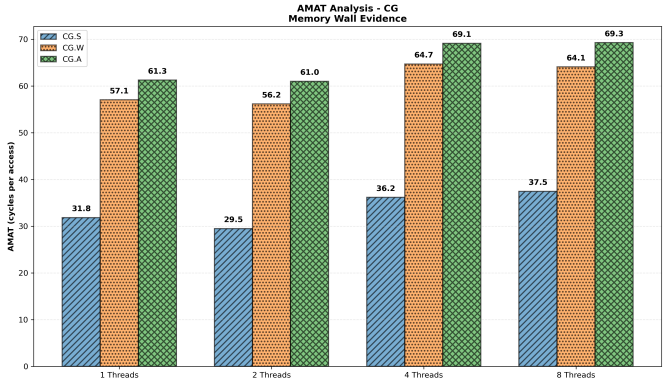

Figure 4. Calculated AMAT per thread and problem size

J. Memory Bandwidth

The Xeon E5603 has a theoretical memory bandwidth of 34.1 GB/s and a cache line size of 64 bytes. Given the high miss rate, memory traffic is significant. Bandwidth demand can be estimated as:

$$\text{BW} = \frac{\text{L1 Misses} \times 64 \text{ bytes}}{\text{Execution Time (s)}} \tag{2}$$

For Class A with 4 threads:

$$\text{BW} \approx 60 \text{ GB/s}$$

This exceeds the theoretical limit of the processor, demonstrating that the system becomes memory-bound. The bottleneck shifts from computation to memory throughput, explaining why increasing threads beyond 4 degrades performance. Note that this bandwidth estimate assumes all L1 misses reach main memory, so it represents a theoretical upper bound. In practice, some accesses are served by L2 or L3 cache.

III. Conclusions

Performance in modern processors is often limited by the memory subsystem rather than computational capacity. Data movement (throughput) plays a critical role in determining overall performance.

This benchmark highlights the importance of spatial locality, memory latency, and system-level overheads such as context switching.

A. Validation on a Modern Architecture

To validate these findings, the benchmark was profiled on an AMD Ryzen 7 5800HS processor.

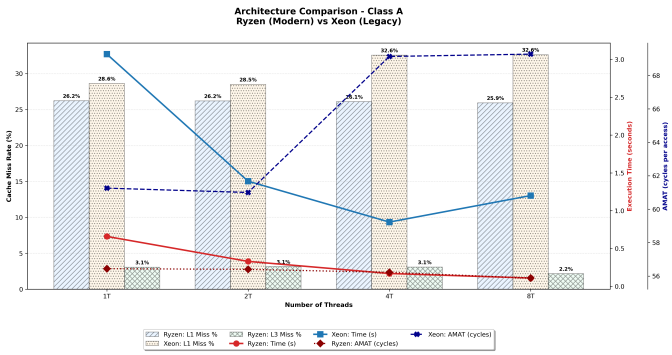

Figure 5. Comparison of Intel Xeon E5603 and AMD Ryzen 7 5800HS

Despite architectural differences, results show that L1 miss rates remain similar, confirming that the irregular access pattern dominates behavior. While the larger 16MB L3 cache on the Ryzen significantly reduces last-level cache misses, improvements in prefetching, memory controller efficiency, SMT, and higher clock frequencies also contribute to better scalability.